

Sec. 4.3. Linux Shells

Linux Shells

Task 1 - Introduction to Linux Shells

The Linux kernel is the core part of the operating system responsible for managing hardware, processes, memory, and system resources. The shell is a command-line interface that allows users to interact with the kernel by executing commands. The shell acts as a bridge between the user and the kernel.

Answer the questions below

| Who is the facilitator between the user and the OS? Shell

Task 2 - How To Interact With a Shell?

Bash is a Unix/Linux command-line shell that allows users to interact with the operating system and automate tasks through scripting. It is widely used by system administrators and cybersecurity professionals for automation, administration, and security operations.

To see your current working directory, you can execute `pwd`, which stands for Print Working Directory, as shown in the terminal below:

```
user@tryhackme:~$ pwd
/home/user
```

However, you can change your directory as well. To do that, you can use `cd` (short for Change Directory), as shown in the terminal below:

```
user@tryhackme:~$ cd Desktop
user@tryhackme:~/Desktop$
```

List Directory Contents

```
user@tryhackme:~$ ls
Desktop Documents Downloads Music Pictures Public Templates Videos
```

Displaying File Contents

```
user@tryhackme:~$ cat filename.txt
this is a sample file
this is the second line of the file
```

The grep command is a very popular command among Linux users.

```
user@tryhackme:~$ grep THM dictionary.txt
The flag is THM
```

Answer the questions below

- | What is the default shell in most Linux distributions? Bash
- | Which command utility is used to list down the contents of a directory? ls
- | Which command utility can help you search for anything in a file? grep

Task 3 - Types of Linux Shells

```
user@tryhackme:~$ echo $SHELL
/bin/bash
```

You can also list down the available shells in your Linux OS. The file `/etc/shells` contains all the installed shells on a Linux system. You can list down the available shells in your Linux OS by typing `cat /etc/shells` in the terminal:

Available Shells

```
user@tryhackme:~$ cat /etc/shells
# /etc/shells: valid login shells
```

```
/bin/sh
/bin/bash
/usr/bin/bash
/bin/rbash
/usr/bin/rbash
/bin/dash
/usr/bin/dash
/usr/bin/tmux
/usr/bin/screen
/bin/zsh
/usr/bin/zsh
```

To switch between these shells, you can type the shell name that is present on your OS, and it will open for you, as can be seen below: Switch Shell

```
user@tryhackme:~$ zsh
tryhackme%
```

If you want to permanently change your default shell, you can use the command: `chsh -s /usr/bin/zsh`. This will make this shell as the default shell for your terminal.

Feature	Bash	Fish	Zsh
Full Name	The full form of Bash is Bourne Again Shell.	The full form of Fish is Friendly Interactive Shell.	The full form of Zsh is Z Shell.
Scripting	It offers widely compatible scripting with extensive documentation available.	It has limited scripting features as compared to the other two shells.	It offers an excellent level of scripting, combining the traditional capabilities of Bash shell with some extra features.

Feature	Bash	Fish	Zsh
Tab completion	It has a basic tab completion feature.	It offers advanced tab completion by giving suggestions based on your previous commands.	Its tab completion capability can be extended heavily by using plugins.
Customization	Basic level of customization.	It offers some good customization through interactive tools.	Advanced customization through oh-my-zsh framework.
User friendliness	It is less user-friendly, but being a traditional and widely used shell, its users are quite familiar and comfortable with it.	It is the most user-friendly shell.	It can be highly user-friendly with proper customization.
Syntax highlighting	The syntax highlighting feature is not available in this shell.	The syntax highlighting is built-in to this shell.	The syntax highlighting can be used with this shell by introducing some plugins.

Answer the questions below

- | Which shell comes with syntax highlighting as an out-of-the-box feature? Fish
- | Which shell does not have auto spell correction? Bash
- | Which command displays all the previously executed commands of the current session? history

Task 4 - Shell Scripting and Components

A **shell script** is a file containing a sequence of commands that can be executed automatically instead of typing commands manually each time.

- Shell scripts help **automate repetitive tasks** and improve efficiency.
- Linux shells such as **Bash** support scripting capabilities.

- Scripts are created using a text editor and commonly saved with the `.sh` **extension** for Bash scripts.
- Scripting is not limited to shells; it can also be done using programming languages like Python, but shell scripting focuses on automating system commands.

Example:

```
#!/bin/bash echo "System check started"
uptime
df -h
```

Running the script executes all commands automatically. The file must be named with an extension `.sh`, the default extension for bash scripts. The following terminal shows the script file creation:

```
user@tryhackme:~$ nano first_script.sh
```

Every script should start from shebang. Shebang is a combination of some characters that are added at the beginning of a script, starting with `#!` followed by the name of the interpreter to use while executing the script. As we are writing our script in bash, let's define it as the interpreter in the shebang. `first_script.sh`

```
#!/bin/bash
```

Variables: A **variable** is used to store a value that can be reused throughout a script. Instead of repeatedly writing complex values such as URLs, file paths, or user input, you store them in a variable and reference the variable name.

- Variables make scripts easier to read, maintain, and automate.
- The `read` command is used to capture user input and store it in a variable.
- The `echo` command is used to display output.

```
# Defining the Interpreter
#!/bin/bash
echo "Hey, what's your name?"
```

```
read name
echo "Welcome, $name"
```

Now, save the script by pressing **CTRL+X**. Confirm by pressing **Y** and then **ENTER**. To execute the script, we first need to make sure that the script has execution permissions. To give these permissions to the script, we can type the following command in our terminal: Execution Permission to Script

```
user@tryhackme:~$ chmod +x first_script.sh
```

To run a shell script located in the current directory, use **./** before the script name. Example:

```
./script.sh
```

- **./** tells the shell to execute the script from the **current directory**.
- Without **./**, the shell searches for the script in directories listed in the **PATH environment variable**.
- If the script is not located in PATH, the shell returns an error.
- The script must have **execute permissions** before running:

```
chmod+x script.sh
```

Then execute:

```
./script.sh
```

This ensures the shell knows the exact location of the script.

```
user@ubuntu:~$ ./first_script.sh
Hey, What's your name?
```

```
John
Welcome, John
```

A **loop** is a programming structure that repeats a set of commands multiple times until a condition is met.

- Loops help automate repetitive tasks.
- Instead of performing the same action manually, a loop can execute commands for multiple items automatically.
- In shell scripting, loops can be used to process lists, files, users, or perform repeated operations.

```
# Defining the Interpreter
#!/bin/bash
for i in {1..10};
do
echo $i
done
```

The first line has the variable `i` that will iterate from 1 to 10 and execute the below code every time. `do` indicates the start of the loop code, and `done` indicates the end. In between them, the code we want to execute in the loop is to be written. The for loop will take each number in the brackets and assign it to the variable `i` in each iteration. The `echo $i` will display this variable's value every iteration. Now, let's execute the script after giving it the execution permission. Script Execution

```
user@tryhackme:~$ ./loop_script.sh
1
2
3
```

The output of the above terminal is cut to `3` numbers only for demonstration. However, when executed according to the script's logic, it would display the numbers from `1 to 10`.

Conditional statements are an essential part of scripting. They help you execute a specific code only when a condition is satisfied; otherwise, you can execute another code. Suppose you want to make a script that shows the user a secret. However, you want it to be shown to only some users, only to the high-authority user. You will create a conditional statement that will first ask the user their name, and if that name matches the high authority user's name, it will display the secret. First, create a new file named `conditional_script.sh`, then enter the code below. Save your file by pressing `CRTL+X`, then confirm with `y` and then `ENTER`.

```
# Defining the Interpreter
#!/bin/bash
echo "Please enter your name first:"
read name
if [ "$name" = "Stewart" ]; then
    echo "Welcome Stewart! Here is the secret: THM_Script"
else
    echo "Sorry! You are not authorized to access the secret."
fi
```

The above script takes the user's name as input and stores it into a variable (studied in the Variables section). The conditional statement starts with `if` and compares the value of that variable with the string `Stewart`; if it's a match, it will display the secret to the user, or else it will not. The `fi` is used to end the condition. Following is the terminal showing the script execution when the user name matches the authorized one defined in the script: `conditional_script.sh`

```
user@tryhackme:~$ ./conditional_script.sh
Please enter your name first:
Stewart
Welcome, Stewart! Here is the secret: THM_Script
```

However, the following terminal shows the script execution when the user name does not match the authorized one defined in the script: `conditional_script.sh`

```
user@tryhackme:~$ ./conditional_script.sh
Please enter your name first:
Alex
Sorry! You are not authorized to access the secret.
```

Comments: A comment is a sentence that we write in our code just for the sake of our understanding. It is written with a # sign followed by a space and the sentence you need to write. For example, let's rewrite the script we discussed in the conditional statements section and add comments to it. Open the `conditional_script.sh` with `nano`, then add the comments starting with a # sign. Save your file by pressing `CRTL+X`, then confirm with `y` and then `ENTER`.

```
# Defining the Interpreter
#!/bin/bash

# Asking the user to enter a value.
echo "Please enter your name first:"

# Storing the user input value in a variable.
read name

# Checking if the name the user entered is equal to our required name.
if [ "$name" = "Stewart" ]; then

# If it equals the required name, the following line will be displayed.
echo "Welcome Stewart! Here is the secret: THM_Script"

# Defining the sentence to be displayed if the condition fails.
else
    echo "Sorry! You are not authorized to access the secret."
```

```
ret."  
fi
```

Answer the questions below

- What is the shebang used in a Bash script? `#!/bin/bash`
- Which command gives executable permissions to a script? `chmod +x`
- Which scripting functionality helps us configure iterative tasks? loops

Task 5 - The Locker Script

A **Linux Locker Script** is a custom shell script used to restrict access to a terminal, file, or system resource until a required condition is satisfied, such as entering the correct password or verification value.

- It is **not a built-in Linux feature**; it is created using scripting.
- It can simulate or implement simple access control mechanisms.
- Common uses include:
 - Cybersecurity training
 - System administration tasks
 - Automation and access control demonstrations

Example use case: A script prompts a user for a password and only allows access if the correct value is provided. Otherwise, access is denied.

In cybersecurity, locker scripts help demonstrate concepts like **authentication, authorization, and conditional logic**.

```
user@tryhackme:~$ ./locker_script.sh  
Enter your Username:  
John  
Enter your Company name:  
Tryhackme  
Enter your PIN:
```

1349

Authentication Denied!!

Answer the questions below

What would be the correct PIN to authenticate in the locker script? 7385

Task 6 - Practical Exercise

We have placed a script on the default user directory `/home/user` of the attached Ubuntu machine. This script searches for a specific keyword in all the files (with `.log` extension) in a specific directory. Become Root User

```
user@tryhackme:~$ sudo su
[sudo] password for user:
root@tryhackme:/home/user#
```

You can make the changes in the script file by keeping in view the following details:

- **Flag:** thm-flag01-script
- **Directory:** /var/log

Hint: Look for empty double quotes `" "` inside the script file and fill them. Make sure not to leave any space between them.

Answer the questions below

Which file has the keyword? authentication.log

Where is the cat sleeping? under the table